Dokumentation - Metallic Delay

Praxismodul Musikinformatik II

Einleitung **2**
Benutzeroberfläche **3**
Implementierung **4**
Die Klasse CombDelay 5
Die Klasse FeedbackFilter 6
Die Klasse Distortion 7
Fazit und Ausblick **8**
References **9**

Einleitung

Im Rahmen dieses Projekts soll ein Audio-Plugin auf Basis des C++ Frameworks JUCE konzipiert und entwickelt werden. Das Plugin trägt den Titel *MetallicDelay* und setzt die Idee um, aus einem Input-Signal einen chaotisch-resonanten metallartigen Sound abzuleiten. Dies soll durch ein kurzes Delay, hohes Feedback und variabler Distortion in der Feedbackschleife erreicht werden.
Sullivan beschreibt ebenfalls ein Vorgehen, das auf dem Karpus-Strong-Algorithmus basiert. Dabei werden in der Feedbackschleife eines Delays Filter und Distortion verwendet, um den gewünschten Klang zu erhalten. (Sullivan, 1990, 26-37)
Durch die nichtlineare Verzerrung im Feedback entsteht ein unharmonischer und metallischer Klang. (Zölzer, 2011, S.101 ff.)

In Abbildung 1 ist ein skizzenhafter Signalfluss zu sehen. Dieser verdeutlicht, dass die Samples des Eingangssignals zunächst in eine Delay-Line geschrieben werden. Aus der Delay-Line werden Samples aus der Vergangenheit gelesen und im Anschluss verzerrt, wodurch neue Frequenzen im Signal entstehen. Um die Klangfarbe des Feedbacks zusätzlich formen zu können, werden Filter eingesetzt. Das entsprechend verarbeitete Sample wird daraufhin erneut in die Delay-Line geschrieben. Je nachdem wie hoch der Feedback-Wert ist, bauen die wiederholten Verzerrungen und Frequenzfilterungen aufeinander auf und bringen bei einem hohen Feedbackwert verstärkt neue unharmonische Frequenzen in das Signal, was den Klang metallischer macht. Am Schluss werden das verarbeitete Signal und das rohe Eingangssignal zusammengerechnet. Das Mischverhältnis wird durch einen Dry/Wet-Wert bestimmt.

Benutzeroberfläche

In Abbildung 2 ist die Benutzeroberfläche des *MetallicDelay* zu sehen, welches von dem skizzierten Signalfluss und den daraus resultierenden Anforderungen für das Plugin abgeleitet wurde. Der Parameter *delay time* bestimmt die zeitliche Verschiebung des verzögerten Signals. Dessen Wertebereich liegt zwischen 0.1 und 10ms. Durch seine sehr kurze Verzögerung wirkt der Delay wie ein Kammfilter, der bestimmte Frequenzen hervorhebt und andere auslöscht. (Zölzer, 2011, S.72)

Der Feedback-Parameter bestimmt, zu welchem Anteil das verzögerte Sample in die Delay-Line geschrieben werden soll und bewegt sich im Wertebereich 0.0 bis 1.0. Die Delay-Time lässt sich über die Parameter *mod rate, mod depth* und *shape* automatisch modulieren. *Mod rate* bestimmt dabei die Frequenz in der sich die Delay-Time ändert und *mod depth* die Stärke der Modulation. Über den *shape*-Parameter lässt sich die Form der Modulation wählen. Zur Verfügung stehen dabei eine Sinus-Welle, sowie eine Dreieck- und eine Rechteckwelle. Zusätzlich gibt es einen “Sample and Hold"-Modus, bei dem zu zufälligen Zeitpunkten zufällige Delay-Time-Werte generiert werden.

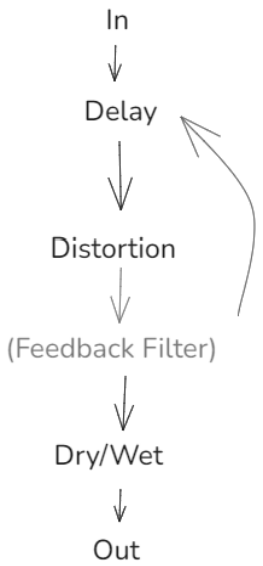

Abb. 1: Signalfluss

Zu den Parametern, die das Feedback beeinflussen, gehören die Cutoff-Frequenzen der Highpass- und Lowpass-Filter, sowie die Verzerrungs- oder Distortion-Parameter. Der *mode*-Parameter legt die Art der Distortion fest. Dabei wählt der Benutzer zwischen Softclip, Hardclip und Wavefold. *Drive* bildet einen abstrahierenden Parameter, der für jede Distortion-Art auf entsprechende Weise die Wirkungsstärke der Verzerrung steuert. Der Wertebereich für *drive* liegt zwischen 0.0 und 1.0. Das Minimum von *drive* ist gleichzusetzen mit keiner Verzerrung und somit keiner Veränderung des Ursprungsspektrums. Je höher *drive*, desto mehr setzt sich der Charakter der jeweiligen Distortion durch.

Out gain legt die letztendliche Lautstärke des Klangs fest und *dry/wet* das Mischverhältnis zwischen dem Ursprungssignal und dem verarbeiteten Signal.

Abb. 2: GUI des *MetallicDelay*

Implementierung

Die Zustände der auf der Benutzeroberfläche sichtbaren Parameter werden in einem *AudioParameterValueTreeState*-Objekt festgehalten. Sowohl der Audio-Thread, als auch der GUI-Thread haben Zugriff auf dieses Objekt, wodurch die Kommunikation zwischen *PluginEditor* und *PluginProcessor* möglich gemacht wird. (JUCE, 2024)

Die einzelnen Elemente, aus denen *MetallicDelay* besteht, werden in der Implementierung so in einzelne Klassen aufgeteilt, dass zusammengehörige Funktionalitäten gekapselt und unabhängig voneinander verändert und eingesetzt werden können. Dementsprechend sind alle Delay-Funktionalitäten in einer Klasse gekapselt, sowie die Funktionalitäten der Filter, Distortion und Modulation.
In der *processBlock*-Methode der *PluginProcessor*-Klasse werden alle Elemente des Effekts entsprechend dem angestrebten Signalfluss zusammengeführt (siehe Abbildung 3). Zunächst werden Samples aus dem Eingangssignal und dem *CombDelay* gelesen. *CombDelay* ist eine Klasse, die die Funktionalitäten des Delay-Mechanismus zusammenfasst. Ein *CombDelay*-Objekt enthält einen *vector* mit vergangenen Samples. Je höher der Wert der Delay-Time, desto weiter in der Vergangenheit liegt das gelesene Sample. Das verzögerte Sample wird vom *distortion*-Objekt je nach aktuellem Distortion-Modus verzerrt. Daraufhin wird das *feedbackFilter*-Objekt eingesetzt, um den Klang des Samples weiter zu formen. Das Sample wird mit dem Feedback-Faktor multipliziert, mit dem Eingangssignal verrechnet und in den *combDelay* geschrieben. Für das letztendliche Ausgangssignal werden das Eingangssignal und das verarbeitete Signal zu entsprechenden Anteilen, die sich aus dem *dry/wet*-Wert ergeben, zusammengerechnet.

Im Folgenden werden die Funktionalitäten der einzelnen Klassen näher beleuchtet.

Die Klasse *CombDelay*

CombDelay sammelt in einem *vector*, der als *delayBuffer* bezeichnet wird, Samples des Eingangssignals, sodass je nach gewählter Delay-Time Samples aus der Vergangenheit gelesen werden können. Die Größe des *vectors* wird bei Initialisierung eines *CombDelay*-Objekts so gewählt, dass jederzeit mindestens auf alle Samples aus der maximalen Delay-Time zurückgegriffen werden kann. Lokal wird in einem *CombDelay*-Objekt zudem die aktuelle Delay-Time festgehalten, welche über die Methode *setDelayMs* gesetzt werden kann. Die in Millisekunden übergebene Größe wird in Samples umgerechnet, um im weiteren Verlauf intuitiver auf benötigte Samples zugreifen zu können. In Abbildung 4 zeigt neben *setDelayMs* zwei weitere wichtige Methoden der Klasse. Die Methode *read* schreibt gelesene Samples in den *delayBuffer* und inkrementiert dabei die Variable *writePos* die den aktuellen Schreibindex im *delayBuffer* repräsentiert. Da der *delayBuffer* als Ring-Buffer fungiert, wird *writePos* auf 0 gesetzt, um zu weit in der Vergangenheit liegende Samples zu überschreiben. In der *read*-Methode wird ausgehend von der aktuellen Schreibposition des *delayBuffers* auf das Sample zugegriffen, was vor der angegebenen Delay-Time in den *delayBuffer* geschrieben wurde.

```
void MetallicDelayAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels  = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    updateParameters(totalNumInputChannels, buffer.getNumSamples());

    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear (i, 0, buffer.getNumSamples());

    for (int channel = 0; channel < totalNumInputChannels; ++channel)
    {
        auto* channelData = buffer.getWritePointer (channel);

        for (int i = 0; i < buffer.getNumSamples(); ++i)
        {
            float in = channelData[i];
            float delayed = combDelays[channel].read();

            float distorted = distortion.processSample(delayed);

            float lPassed = feedbackFilter.lowpass.processSample(distorted);
            float hpassed = feedbackFilter.highpass.processSample(lPassed);
            float wet = hpassed * combDelays[channel].getFeedback();

            float out = in + wet;

            combDelays[channel].write(out);
            channelData[i] = (in * dryGain + wet * wetGain) * gainLinear;
        }
    }
}
```

Abb. 3: processBlock-Methode

```cpp
void CombDelay::setDelayMs(float ms)
{
    delayMs = ms;

    float samples = ms * 0.001f * sr;
    delaySamples = juce::jlimit(1, bufferSize - 1, (int)samples);
}

float CombDelay::read()
{
    int readPos = writePos - delaySamples;
    if (readPos < 0)
        readPos += bufferSize;

    return delayBuffer[readPos];
}

void CombDelay::write(float output)
{
    delayBuffer[writePos] = output;
    writePos = (writePos + 1) % bufferSize;
```

Abb. 4: Methoden aus der Klasse *CombDelay*

Die Klasse FeedbackFilter

Die Klasse *FeedbackFilter* enthält die aktuellen Zustände des High- und Lowpassfilters, die genutzt werden, um hohe oder tiefe Frequenzen der Samples in der Feedback-Schleife herauszufiltern. Dazu gehören die gewählten Cutoff-Frequenzen für jeweils High- und Lowpassfilter und Funktionen, um diese und die damit zusammenhängenden Filterkoeffizienten zu aktualisieren.
Zur Umsetzung der Filter werden Hilfsklassen aus dem JUCE-Modul juce::dsp zur Hand genommen. Um von dessen Funktionalitäten Gebrauch zu machen, werden in der *prepare*-Methode der *FeedbackFilter*-Klasse die benötigten Informationen aus dem aktuellen Audio-Kontext übergeben, sodass die JUCE-Filter-Klassen diese im weiteren Verlauf für die Verarbeitung der Samples verwenden können.

```cpp
void FeedbackFilter::updateLowpass(float cutoffHz)
{
    auto coeffs = juce::dsp::IIR::Coefficients<float>::makeLowPass(sampleRate, cutoffHz);
    *lowpass.coefficients = *coeffs;
}

void FeedbackFilter::updateHighpass(float cutoffHz)
{
    auto coeffs = juce::dsp::IIR::Coefficients<float>::makeHighPass(sampleRate, cutoffHz);
    *highpass.coefficients = *coeffs;
}
```

Abb. 5: Methoden aus der Klasse *FilterFeedback*

Abbildung 5 zeigt Methoden, die verwendet werden, um bei Bedarf die Koeffizienten der beiden Filter anzupassen. In der *processBlock*-Methode im *PluginProcessor* kann dann *processSample* auf den High- und Lowpass-Objekten der *FeedbackFilter*-Klasse aufgerufen werden, um eingelesene Samples zu filtern (siehe Abbildung 3).

Die Klasse Distortion

Audioeffekte wie Overdrive oder Distortion erzeugen im Eingangssignal neue spektrale Inhalte, die harmonisch sowie unharmonisch sein können. Die spektrale Verzerrung wird durch die Nichtlinearität dieser Effekt-Algorithmen hervorgehoben. Lineare Systeme können lediglich Amplituden oder Phasen eines Sinusoiden verändern. Wohingegen nichtlineare Systeme eine Summe von Sinusoiden zurückgeben. Die unharmonischen Komponenten, die durch nichtlineare Systeme erzeugt werden, resultieren aus gegenseitiger Amplitudenmodulation der erzeugten Obertöne. Eine weitere Quelle für unharmonische Obertöne in digitalen Systemen sind Artefakte durch Aliasing.
In *MetallicDelay* sind drei Distortion-Typen verfügbar, um verschiedene spektrale Verzerrungseffekte erzielen zu können. Beispielsweise unterscheiden sich die Distortion-Typen Softclip und Hardclip in der psychoakustischen Wahrnehmung. Verzerrung durch Softclipping wirkt wärmer und sauberer als Verzerrung durch Hardclipping. Softclipping kann durch eine Funktion der Form *tanh(x * g)* beschrieben werden. Dabei ist *x* das Signal und *g* ein Pregain-Faktor der bestimmt, wie schnell die Werte der Funktion sich 1 annähern. Die Funktion bleibt dabei jedoch glatt und es braucht weniger Obertöne, um das Signal zu rekonstruieren. Beim Hardclipping wird das Signal ab einem bestimmten Threshold “abgeschnitten”, wodurch die Funktion unglatt wird, was dazu führt, dass unendlich viele Obertöne benötigt werden, um das Signal zu beschreiben, was in digitalen Systemen zu Aliasing und zu weiteren unharmonisch Obertönen führt. Dadurch wirken die spektralen Inhalte aggressiver und härter. (Reyes & Alberto, 2024)
Der dritte Distortion-Typ, der zur Auswahl steht, ist das Wavefolding. Dabei werden Werte eines Signals, die über einen bestimmten Threshold hinausgehen, an der Threshhold-Grenze reflektiert. Durch das abrupte Abknicken an der Threshold-Grenze entstehen ähnlich wie beim Hardclipping starke spektrale Erweiterungen. Im Gegensatz zum Hardclipping, entstehen dabei aber nicht nur ungerade Harmonien, sondern auch gerade, was letztendlich zu einem breiteren und komplexeren Spektrum führt. (Zölzer, 2011, S. 124ff.)

Abbildungen 6a und 6b zeigen Methoden, die je nach ausgewähltem Distortion-Typ entsprechende Berechnungen ausführen. Zuvor wird der Wert des abstrahierenden Parameters *drive* über die Methode *shapeDrive* so skaliert, dass er je nach Distortion-Typ unterschiedlich starke Auswirkung hat. Die Methode *wavefold* nutzt eine Dreiecksfunktion, um eingehende Werte an den Thresholdgrenzen 1.0 und -1.0 zu reflektieren.

```
float processSample(float x) {
    float shaped = shapeDrive();
    float preGain = juce::jmap(shaped, 0.0f, 1.0f, 1.0f, 10.0f);
    float threshold = juce::jmap(shaped, 1.f, 0.1f);
    float out = 0.f;

    switch (mode)
    {
    case Distortion::Softclip:
        return std::tanh(x * preGain);
    case Distortion::Hardclip:
        return juce::jlimit(-threshold, threshold, x * preGain);
    case Distortion::Wavefold:
        return wavefold(x * preGain);
    default:
        break;
    }
}
```

Abb. 6a: *processSample* in der Klasse *Distortion*

```
float shapeDrive()
{
    switch (mode)
    {
    case Mode::Softclip:  return std::pow(drive, 2.5f);
    case Mode::Hardclip:  return std::pow(drive, 2.0f);
    case Mode::Wavefold:  return std::pow(drive, 3.5f);
    }
}

float wavefold(float x) {
    return std::abs(std::fmod(x + 1.0f, 4.0f) - 2.0f) - 1.0f;
}
```

Abb. 6b: *shapeDrive* und *wavefold*

Bisher enthält die Klasse *Distortion* noch keine Funktionalitäten, die Anti-Aliasing umsetzen. Man könnte sich bezüglich Anti-Aliasing in nächsten Iterationen in der Weiterentwicklung überlegen, ob man den chaotischen Charakter, der durch die unharmonischen Frequenzen entsteht, beibehalten möchte oder ob man dem Benutzer zumindest die Möglichkeit geben möchte ein harmonischeres spektrales Bild zu erhalten.

Fazit und Ausblick

In diesem Projekt wurde ein Plugin entwickelt, dessen Hauptmechanismus eine Feedbackschleife in einem kurzen Delay ist, die Effekte wie Distortion und Filter integriert. Der Nutzer kann den metallischen Charakter des dadurch entstehenden Klangs durch gegebene Parameter formen. Das Plugin kann in DAWs verwendet werden und durch die Definition eines *AudioParameterValueTreeStates* auch automatisiert werden.
Einige Funktionalitäten können im weiteren Verlauf der Plugin-Entwicklung ausgebaut werden. Zum Beispiel kann bisher nur die Delay-Time mit dem zur Verfügung gestellten LFO moduliert werden. Jedoch könnte auch die Modulation weiterer Parameter ein komplexes Sounddesign ermöglichen.
Da die Distortion als Hauptelement in der Feedbackschleife zum metallischen Charakter des resultierenden Klangs beiträgt, würde es sich lohnen, entsprechende Funktionalitäten zu erweitern. Allein die Implementierung weiterer Distortion-Typen könnte zu einer größeren Bandbreite an Möglichkeiten zur Generierung neuer spektraler Inhalte verhelfen. Die Umsetzung von Anti-Aliasing würde dem Benutzer mehr Kontrolle über den Klangcharakter geben.

References

JUCE. (2024). *Tutorial: Adding plug-in parameters*. JUCE. Retrieved March 9, 2026, from https://juce.com/tutorials/tutorial_audio_parameter/

Reyes, V., & Alberto, M. (2024). Psychoacoustics of Soft Clipping: Validation through Time and Frequency Analysis. *Audio Engineering Society Latin American Convention 2024*, *1*(1). https://doi.org/10.15124/yao-3bc6-4859

Sullivan, C. R. (1990). Extending the Karplus-Strong Algorithm to Synthesize Electric Guitar Timbres with Distortion and Feedback. *Computer Music Journal*, *14*(3), 26-37. https://doi.org/10.2307/3679957

Zölzer, U. (Ed.). (2011). *DAFX: Digital Audio Effects*. Wiley. ISBN: 978-0-470-66599-2